

DATIX AI
Project Documentation

PROJECT 2

Camera-Based Collision Prevention System

LionChief Dual BLE Train Control — Single Track

Client	Peter (Piotr) — USA
Developer	Datix AI, Pakistan
Version	v10.0 — July 2026
Repository	github.com/datixai/lionchief-controller-AI-ESP32

1. Project Overview

Project 2 is a camera-based automatic collision prevention system for Peter's model train layout. Two LionChief Bluetooth trains run on the same outer loop track. A USB ceiling camera watches both trains. Python software detects each train's position in real time and automatically controls Train B (ORANGE) speed to maintain a safe gap behind Train A (BLACK).

What the System Does

- Detects both trains using MOG2 motion-based computer vision
- Tracks each train continuously even through turns and speed changes
- Automatically slows Train B (ORANGE) when it gets too close to Train A (BLACK)
- Automatically speeds up Train B when the gap is too large
- Detects when trains swap positions and handles it with ESCAPE mode
- Stops Train B in DANGER zone; Train A keeps running to open the gap
- Auto-pauses when tracker is lost; resumes when both trains re-acquired

System Components

Track	Single outer loop — both trains share the same track
Train A	BLACK Train — front leader (BLE controlled)
Train B	ORANGE Train — rear follower (gap controlled)
Camera	USB ceiling camera (index 1) — overhead view of layout
Computer	Peter's Windows PC running Python scripts
Software	Python + OpenCV + Bleak (BLE library)
Folder	single_trackDualBT

BLE Train MAC Addresses

BLACK Train (A)	60:FA:11:E3:94:C9 Name: LC-1-1-09F8-60FA
ORANGE Train (B)	60:F9:FB:49:94:C9 Name: LC-1-1-09F9-60F9

2. PC Setup and File Paths

Peter's PC — Folder Structure

```
C:\Users\peter\Documents\Linchief_train\
├── single_trackDualBT\
│   ├── main.py           ← main program
│   ├── train_detector.py ← tracking system
│   ├── ble_controller.py ← BLE connection
│   ├── speed_controller.py ← gap-to-speed logic
│   ├── config.py         ← all settings
│   ├── calibrate.py      ← distance calibration
│   ├── orange.py         ← orange train test script
│   ├── table_mask.json   ← saved table boundary
│   └── track_path.json   ← saved track path (optional)
```

i If Peter's new PC username is Post-war, the path becomes C:\Users\Post-war\Documents\Linchief_train\single_trackDualBT

Prerequisites — One-Time Installation

1	Install Git	Download from https://git-scm.com/download/win — keep all default settings — restart PC after install
2	Install Python	Download Python 3.11 from https://python.org — IMPORTANT: check "Add Python to PATH" during install
3	Install VS Code	Download from https://code.visualstudio.com (optional — any terminal works)
4	Clone Repo	Open terminal: <code>cd C:\Users\peter\Documents</code> then <code>git clone https://github.com/datixai/lionchief-controller-AI-ESP32.git</code> Linchief_train
5	Create venv	<code>cd Linchief_train\single_trackDualBT</code> then <code>python -m venv .venv</code>
6	Activate venv	Run: <code>Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned</code> then <code>.venv\Scripts\activate</code>
7	Install packages	<code>pip install opencv-contrib-python bleak numpy</code>

Getting Updates from GitHub

Whenever new code is pushed from Datix AI, Peter runs:

```
cd C:\Users\peter\Documents\Linchief_train
.venv\Scripts\activate
git pull
cd single_trackDualBT
python main.py
```

⚠ If git says "Everything up to date" but code looks old — Datix AI may not have pushed yet. Contact Ahmed to confirm the push was done.

3. Running the Program

Every Session — Start Commands

Open PowerShell or VS Code terminal and run these in order:

```
cd C:\Users\peter\Documents\Linchief_train
.venv\Scripts\activate
cd single_trackDualBT
python main.py
```

i The (.venv) prefix appears in the terminal when virtual environment is active. If you do not see it, run the activate command again.

What Happens at Startup

1	BLE Connects	Both trains are found and connected via Bluetooth. Watch terminal for "[OK] TrainA Connected" and "[OK] TrainB Connected"
2	Camera Warms Up	40 frames captured silently so auto-exposure settles — prevents camera flicker on open
3	Table Mask Loads	Saved table boundary loaded automatically. Terminal shows "[OK] Table mask LOADED"
4	Window Opens	Camera view appears. Screen shows STEP 2 — draw box on BLACK Train
5	Draw Boxes	User draws boxes on both trains (see Section 4)
6	Y/N Prompt	After both trains selected — system asks to start
7	Running	Auto mode controls Train B speed based on gap to Train A

Startup Checklist — Before Running

- Both trains are on the track and powered ON
- Phone Bluetooth is OFF (it can steal the BLE connection)
- Camera is plugged in and ceiling-mounted with full track view
- Trains are separated by a reasonable distance (not touching)
- Table lamp / lighting is ON — camera needs good light

4. Drawing Boxes — Step by Step

This is the most important part. The boxes tell the system where each train is. They must be drawn correctly every session.

Step 1 — Table Boundary (First Run Only)

The table boundary tells the system to ignore anything outside the track area. People walking near the table will NOT disturb the tracker.

- Press T key to enter table drawing mode
- Hold left mouse button and drag a large rectangle around the entire track oval
- Release mouse — boundary is saved automatically to table_mask.json
- Next runs: loads automatically — no need to redraw unless camera is moved

i Draw the boundary TIGHT around the track oval only — not the whole table. This keeps people standing at the table edge outside the detection zone.

Step 2 — Draw Box on BLACK Train (Front Leader)

- Screen shows: STEP 2 -- DRAG a box around BLACK Train (front leader)
- Find the BLACK train on the camera view
- Hold left mouse button, drag a box around the BLACK train body
- Release mouse
- Screen flashes: BLACK Train locked -- now select ORANGE Train
- A WHITE/GRAY tracking circle appears on the BLACK train

Step 3 — Draw Box on ORANGE Train (Rear Follower)

- Screen shows: STEP 3 -- DRAG a box around ORANGE Train (rear follower)
- Find the ORANGE train on the camera view
- Hold left mouse button, drag a box around the ORANGE train body
- Release mouse
- Screen flashes: ORANGE Train locked -- both tracking!
- An ORANGE tracking circle appears on the ORANGE train

Step 4 — Y / N Confirmation

Y	AUTO mode — gap detection automatically controls ORANGE train speed
N	MANUAL mode — you control both train speeds yourself

If a Box Goes Wrong — Reselect

- Press A key → drag a new box on the BLACK train
- Press B key → drag a new box on the ORANGE train
- Press T key → redraw the table boundary

5. Keyboard Controls

BLACK Train (Front — Train A)

[	Decrease BLACK train speed by 1
]	Increase BLACK train speed by 1
F1	Stop BLACK train immediately
F2	Resume BLACK train at last speed

ORANGE Train (Rear — Train B)

1 - 7	Set ORANGE train to exact speed (1=slowest 7=full power)
+ or =	Speed up ORANGE train by 1
- or _	Slow down ORANGE train by 1
Arrow Up	Speed up ORANGE train by 1
Arrow Down	Slow down ORANGE train by 1
S	Stop ORANGE train
R	Resume ORANGE train at last speed

Shared Controls

M	Toggle AUTO mode ↔ MANUAL mode
E	Emergency stop — BOTH trains immediately
A	Re-select BLACK train (drag new box)
B	Re-select ORANGE train (drag new box)
T	Redraw table boundary
K	Toggle track path recording (optional)
H	Horn — both trains
L	Lights — both trains
Q or ESC	Quit program

6. Speed Zones — How Auto Control Works

The system measures the pixel gap between the two train tracking circles (surface to surface). Based on this gap it assigns one of five zones and controls Train B speed accordingly.

Zone	Gap	Train B Speed	Action
DANGER	< 100 px	STOP (0)	Train B stops immediately. Train A keeps running to open the gap.
WARNING	100–160 px	Speed 2 (min)	Train B slows to minimum speed.
CAUTION	160–230 px	Speed 3	Train B reduces speed.
SAFE	230–320 px	User speed	Train B follows at the user-set speed.
FAR	> 320 px	Speed 7 (max)	Gap too large — Train B catches up at full speed.

PD Controller — Closing Rate

The system does not just react to the current gap. It also measures how fast the gap is CLOSING per frame. If trains are approaching quickly, it brakes earlier. This is called a PD (Proportional-Derivative) controller. BLE latency of 150ms is also compensated — commands are sent slightly early to account for the response delay.

7. Configuration Settings (config.py)

All tuning values are in config.py. No other file needs editing for normal operation.

Train BLE Settings

TRAIN_A_MAC	60:FA:11:E3:94:C9 (BLACK train)
TRAIN_A_NAME	LC-1-1-09F8-60FA
TRAIN_B_MAC	60:F9:FB:49:94:C9 (ORANGE train)
TRAIN_B_NAME	LC-1-1-09F9-60F9

Camera Settings

CAMERA_INDEX	1 (USB ceiling camera — change to 0 if wrong camera opens)
DISPLAY_W / DISPLAY_H	960 x 540 (fixed display size — never change these)
CAMERA_WARMUP_FRAMES	40 (frames read silently at startup to settle auto-exposure)

Tracking Sensitivity

SEARCH_RADIUS	120 px — increase if fast train loses tracking
MIN_BLOB_AREA	60 px — decrease if trains not detected, increase if false detections
MOG2_VAR_THRESHOLD	50 — increase to ignore more background noise
STOPPED_DIFF_THR	18 — reference patch sensitivity for stopped train detection

Speed Settings

DEFAULT_SPEED_A / B	5 (starting speed for each train)
Speed 1 byte	0x04 (4/28 = ~14% of max power)
Speed 7 byte	0x1C (28/28 = 100% full power)
BLE_LATENCY_MS	150ms (time between command sent and train responds)

i Speed commands are now mapped across the full LionChief range (0x04 to 0x1C). Previous versions only used 0x01-0x07 which was only 25% of actual motor power. Speed 5 now equals approximately 71% power.

8. Known Problems and Limitations

1. Tracking Loses Train on Sharp Turns

MOG2 background subtraction detects motion blobs. On tight curves, the train may move faster than the search radius can follow, causing the tracker to briefly lose position. The 90-frame hold and reference patch matching help recover it.

- Workaround: increase SEARCH_RADIUS in config.py from 120 to 150
- Workaround: press A or B to re-draw the box on the lost train

2. Both Trains Touching — Blob Merges

When the two trains are physically touching or overlapping in camera view, MOG2 sees one large blob instead of two. The system cannot reliably separate them. The blob mutex prevents one tracker stealing the other's blob, but a merged blob still confuses both trackers.

⚠ This is a fundamental limitation of motion-based tracking. The only 100% reliable solution is ArUco markers — small printed squares on each train roof that give the camera unique IDs for each train.

3. ESCAPE Mode Oscillation

When Train A is detected behind Train B (positions swapped on the loop), the system enters ESCAPE mode and speeds up Train B. If the velocity estimate flips rapidly, ESCAPE mode turns on and off every frame, flooding the BLE connection with alternating speed commands. This causes BLE disconnection.

- Partial fix: 15-frame hysteresis before ESCAPE activates
- Partial fix: 1-second rate limit on Train A speed commands
- Known issue: still occurs occasionally on tight curves

4. People Moving Near Table

MOG2 detects all motion inside the table boundary. If a person reaches across the table within the tracking boundary, the motion blob from their arm can pull a tracker away from the train.

- Fix: draw the table boundary TIGHT around the track oval only
- Fix: press T to redraw the boundary closer to the track

5. Train Stops on Hill

At full speed the trains can struggle on the uphill curve section. The gap control may reduce Train B speed exactly when it needs more power for the climb.

- Fix: manually increase Train B speed with] key before the hill section
- Fix: use MANUAL mode on hill sections

6. BLE Range

BLE reliable range is approximately 10-15 meters indoors. The current layout is within range. A larger layout (20+ meters) may experience disconnections at the far end.

9. Orange Train Standalone Test (orange.py)

A separate test file is included to test the ORANGE train on its own — without the BLACK train and without collision prevention. Use this to:

- Verify the ORANGE train BLE connection works
- Test speed commands at full range
- Test tracking independently

Running the Test

```
cd C:\Users\peter\Documents\Linchief_train\single_trackDualBT
.venv\Scripts\activate
python orange.py
```

The program opens the camera, connects to the ORANGE train via BLE, and shows a tracking window. Drag a box on the ORANGE train to start tracking. Speed keys 1-7 control speed directly.

i If ORANGE train connects and responds to speed commands in orange.py but not in main.py, the problem is in the dual BT logic — not in the train or BLE connection.

10. Troubleshooting10. Troubleshooting

Problem	Solution
Train not connecting	Power the train ON. Turn phone Bluetooth OFF. Wait 15 seconds then try again.
python is not recognized	Python not installed or not added to PATH. Reinstall Python, check "Add Python to PATH".
git is not recognized	Git not installed. Download from git-scm.com and restart PC after install.
Camera opens wrong device	Change CAMERA_INDEX in config.py from 1 to 0 (or 2).
Tracking boxes wrong position	Window must be at 960x540. Do not resize the window. Delete table_mask.json and redraw.
Both trains stop and wont restart	DANGER zone deadlock. Press E to emergency stop both. Re-draw boxes. Trains were too close.
DANGER fires constantly	Trains initialized too close. Move trains further apart before drawing boxes. Grace period is 5 seconds.
Train B not responding	Run orange.py to test ORANGE train alone. If it works there, problem is in dual BT logic.
git pull says everything up to date	Ahmed has not pushed new code yet. Wait for confirmation from Datix AI.
Camera flickers on open	Normal — 40 warm-up frames are captured. Wait 2-3 seconds after running for flicker to stop.

Document prepared by Datix AI | Ahmed Ali | July 2026
For support contact Ahmed Ali — Datix AI Pakistan